MacFAST – Scalable Adaptive Testing Platform
Design and V&V Document
January 23, 2026

Group 10 – Capstone Project

Full Name	Email Address (McMaster)	Student Number
Dai Ishida	ishidad@mcmaster.ca	400435830
Mohib Ur Rehman Siddiqui	siddim93@mcmaster.ca	400415895
Jack Fountain	fountj1@mcmaster.ca	400333572
Laeek Ahmed Shaikh	shaikl3@mcmaster.ca	400358738
Raj Rehman	moham50@mcmaster.ca	400359821
Peter Wardell	wardelp@mcmaster.ca	400399034
Jwalit Bhavinkumar Miniwala	miniwalj@mcmaster.ca	400357388

Table of Contents

Version 3

Purpose 4

Component diagram 4

Relationships 4

Component descriptions 5

- Frontend 5
- Adaptive Testing Framework 6
- Backend 7
- Database 8
- UTS Servers 9
- Microsoft Authentication 9

Details on user interface (Figma link here) 10

Verification and Validation Plan 12

1. Frontend 12
   - 1.1 Question Page 12
   - 1.2 Course, Unit, and Subtopic Details 12
   - 1.3 Statistics Page 13
2. Backend 13
   - 2.1 Question File Parser 13
   - 2.2 Adaptive Test Framework 14
   - 2.3 User Data Compilation 14
3. Authentication, and Database 15
   - 3.1 Postgres Database 15
   - 3.2 Authentication and Access Control (M365 SSO) 15
4. Cross-System Validation – Pilot Phase 16

Appendix 16

UI/ UX 17

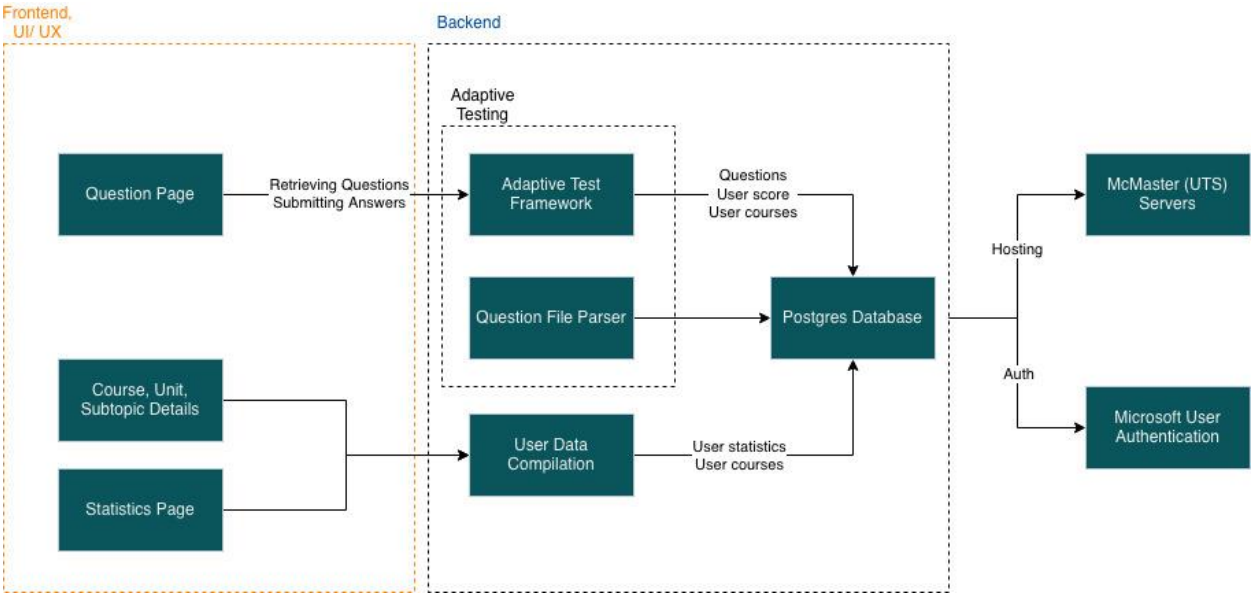
Version

Version	Date	Description	Authors
0	January 23, 2026	Initial Draft	Everyone

Purpose

This document presents the initial design of the system and outlines how verification and validation will be used throughout development to ensure that the design satisfies its requirements and intended user needs. MacFAST is a web-based formative assessment study tool that uses adaptive testing techniques to adjust question difficulty based on student performance, helping them practice course material, receive feedback, and more effectively identify knowledge gaps.

Component diagram



Relationships

Requirement to meet	*Components responsible*
Use McMaster's SSO for authentication/student login	Microsoft Authentication
An adaptive testing framework that will consider the student's "ability score", calculated difficulty of the question, and what topic they are currently studying to determine which question will be asked next.	Adaptive testing framework Database
Support ~7000 total users and at least a significant fraction of concurrent users	Database UTS Servers
Ability to upload test banks/ questions in multiple formats containing multiple types of questions from multiple sources and parse the questions from them • CSVs from A2L quiz exports • Word Documents • HTML files from Scantron	Backend

• Crowdmark exports • Ability to parse LaTeX within each of the above	
The ability to set up classes and assign students to those classes. Students will only then access the tests for that class	Frontend Backend
Students are only able to view their own information, ability score, and progress. • Comparisons to the class average in terms of correctness, and time taken on questions • Data on how much of a certain topic has been covered • Data on which topics the student is struggling with	Frontend Backend Adaptive Testing Framework
Instructors can view each student's information, ability score, and progress as well as comment on questions with other instructors	Frontend Backend
The ability to recognize "bad" questions through flags and analytics	Backend Database

Component descriptions

Frontend

Normal behavior	The frontend provides all-user facing functionalities allowing students to login, select courses, take tests, monitor their progress, see results and statistics, and allowing instructors to upload and edit questions, create units/subtopics and update course dashboards
API	The frontend has several pages (paths) which allow users to interact with the platform: `/` - Home page displays user's enrolled courses (`Course[]`) fetched via `GET /api/courses/`. Renders course cards with progress indicators. `/auth/` - Auth0 authentication related pages `/courses/[courseCode]/coursepage` - Course detail page with tabs (Practice Problems, Learning Objectives, Statistics, Saved Questions). Fetches Course object containing `units[]` and `subtopics[]` with ability scores. Displays hierarchical unit/subtopic navigation. `/courses/[courseCode]/dashboard` - Admin-only dashboard with tabs for Questions management and Class List. Questions tab allows file upload (`FormData`) and displays question bank (`Question[]`).

	`/courses/[courseCode]/[unitName]/[subtopicName]/test` - Adaptive test interface. Fetches `TestQuestion` objects via POST
Implementation	Built with Next.js (App Router), TypeScript, React functional components, and Shadcn UI components styled with Tailwind CSS. Authentication is handled using Auth.js
Undesired behavior	Unable to handle errors from the backend, poor responsiveness or layout issues on smaller screens, lacking accessibility features

Adaptive Testing Framework

Normal behavior	The adaptive testing framework selects and sequences questions for each student based on their estimated ability and the characteristics of available questions. Every time a student answers a question, the framework updates the student's ability estimate (using ITR - Item Response Theory) and selects the next question with appropriate difficulty.
Implementation	A Django app on the backend. Uses Django Views for the business logic at each of the below endpoints and serializers to convert data to and from requests. When a user wants to begin studying a new subtopic, the frontend will get a `TestSession` from the backend and use this to determine the question to show the user. The action they take will be recorded (question was skipped, answered, and whether it was answered correctly) The backend logic also records the question attempts in the `QuestionAttempt` table which can be compiled to provide useful info to the user.
Undesired behavior	Poor question selection due to incorrect IRT parameterization or bias in ability estimation resulting in students being stuck with questions that are too easy or too hard.

API

A little more in depth here because this is an important part of our app.

`POST /api/core/adaptive-test/next-question`	
Params	subtopic_id: The public_id of the subtopic to fetch the next question from
Returns	Question | None - public_id: UUID - content: string - options: array

Retrieves a question for the user

POST /api/core/adaptive-test/submit-answer	
Params	subtopic_id: The public_id of the subtopic to fetch the next question from Option_id: the selected option
Returns	Correct_option_id: UUID Explanation: string
Submits a user's response. Requires only subtopic_id and selected option_id, as the current question they are on is stored in the backend. Returns the id of the correct option and the explanation stored in the backend for why that option is correct.	

POST /api/core/adaptive-test/skip-question	
Params	subtopic_id: The public_id of the subtopic that the user is studying
Returns	Question | None - public_id: UUID - content: string options: array

Method	Endpoint: api/	Description
`GET`	`test-session?subtopic=:subtopic_id`	Get information about the specified test session.
`PUT` `PATCH`	`test-session?subtopic=:subtopic_id`	Create or update a test session.
`DELETE`	`test-session?subtopic=:subtopic_id`	Delete the test session.

Backend

Normal behavior	The backend exposes all core business logic and data access for MacFAST. It authenticates requests (via SSO/JWT), enforces role-based authorization, manages courses, classes, users, questions, adaptive sessions, and analytics, and serves structured JSON for the frontend. It validates inputs, applies adaptive logic, and persists all relevant state to the database.
Implementation	Implemented using Django with a modular structure comprised of the following sub-components or "apps": • core: shared utilities (for the questions model and health check, other base models). • courses: courses and enrolment related models and endpoints.

	• Analytics: analytics models and data storage for `QuestionAttempt` and `UserTopicAbilityScore` • sso_auth: integration with Microsoft SSO/JWT, user model and roles. • MacFAST: main project configuration, URLs, settings.
Undesired behavior	• Race conditions under high load • Failures during question parsing leading to partial or inconsistent question records. • Scaling failures if load balancer for multiple instances is not setup correctly to handle heavy load

API

Method	Endpoint: api/	Description
`GET, POST`	`courses/`	List all courses (Admins) or enrolled courses (Students)
`GET, PUT, PATCH, DELETE`	`courses/:course_code/`	View Course Details
`GET, POST`	`courses/:course_code/units/`	Nested: List units in this course
`GET, POST`	`courses/:course_code/enrolments/`	List students specifically in this course
`GET, POST`	`units/`	Global List.
`GET, PUT, PATCH, DELETE`	`units/:unit_id/`	Retrieve. Shows details + Subtopics list
`GET, POST`	`units/:unit_id/subtopics/`	List subtopics in this unit
`GET, POST`	`subtopics/`	Global List
`GET, PUT, PATCH, DELETE`	`subtopics/:subtopic_id/`	See method name
`GET, POST`	`subtopics/:subtopic_id/questions/`	List questions for this subtopic
`GET, POST`	`questions/`	Global List
`GET, PUT, PATCH, DELETE`	`questions/:question_uuid/`	Retrieve, includes Options & Images
`GET, POST`	`questions/:question_uuid/options/`	List options
`GET`	`subtopic-score/:subtopic_id`	Provides the users score as text.
`GET, PUT, PATCH, DELETE`	`test-session?subtopic=:subtopic_id`	Returns data about the active test session for the user.

Database

Normal behavior	The database persistently stores all structured data for MacFAST: users, courses, classes, question banks, adaptive session state, and analytics. It allows the backend to read/write data in line with ACID principles
API	The backend uses the Django ORM to read and write relational data Inputs are create/update/delete operations over models such as users, courses, classes, questions etc.

	Outputs are query results (model instances, lists, and aggregates) that the backend turns into API responses.
Implementation	Run as a Docker container using the official PostgreSQL image. The container exposes a database port which the Django backend connects to using a username and password supplied via environment variables.
Undesired behavior	• Data inconsistency if migrations are not applied correctly across environments • Risk of data loss if backups and recovery are misconfigured • Slow queries under heavy load

UTS Servers

Normal behavior	Hosting the production and development instances of the frontend, backend, and database. Provides a secure, university-managed environment with network, storage, and security controls that comply with McMaster's policies.
API	The VMs expose application ports (e.g., HTTP/HTTPS) that will be mapped to the MacFAST domain.
Implementation	The servers have been set up by the UTS team where we will install and run the application containers possibly orchestrated using Kubernetes
Undesired behavior	• Resource exhaustion under high concurrent usage, leading to slow responses or timeouts. • Misconfigured firewall or reverse proxy blocking legitimate traffic or SSO callbacks.

Microsoft Authentication

Normal behavior	Provides Single Sign-On for MacFAST users using the university's identity provider. Users log in with their existing McMaster credentials, and no passwords are stored within MacFAST. After a successful login, the frontend and backend receive tokens that identify the user and their roles.
API	We do not have our tenant ID yet, using {tenant} as a placeholder for now. Authorization endpoint: https://login.microsoftonline.com/{tenant}/oauth2/v2.0/authorize Token endpoint: https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token JWKS_URI: https://login.microsoftonline.com/%7btenant%7d/discovery/v2.0/keys

	Userinfo endpoint: https://graph.microsoft.com/oidc/userinfo
Implementation	Our two confidential Microsoft Entra clients will be setup and supported by UTS to match our required configurations.
Undesired behavior	• Misconfigured credentials (client ID/secret or issuer URL) causing login failures across the system. • Token validation errors (expired tokens, incorrect key rotation handling) causing failures. • Security issues if tokens are logged, stored insecurely, or transmitted

Details on user interface (Figma link here)

* Our figma is becoming as outdated as we continue to progress and iterate on designs throughout development, but it is being updated to the newer look.

- Describe the UI design
  - Focused on card-based design, a UI style that is extremely common nowadays being used by companies like Instagram, Spotify, Amazon and just about every dashboard interface (Mosaic!).
  - Card-based design makes it extremely easy to look at a variety of information extremely quickly, especially if proper titles are used. A good example of this is the user's homepage where they can view all the courses that are available to them on MacFast, they only have to look across their distinct cards and the card's title to find their course quickly.
    See Fig. 1
  - For the learning objectives and practice problem sections still focused on card-based design but also incorporated visual hierarchy more intentionally where each card represents a unit that can be expanded into a list structure of subtopics.
    See Fig. 2, Fig. 3
- **user interactions** (Speaking to the professor he mentioned to focus on key actions)

Interaction	**View a course page and it's subtopics**
Pre-condition	User is logged in on their homepage
Screenshot of pre-condition	See Fig. 1
Action	User clicks on the 'details' button or course name on a course card on their homepage.
Flow	1. Frontend redirects the user to the loading course page 2. Frontend sends a request with the user's auth token to the backend to `/api/courses/{course_code}/` 3. Backend receives request and validates the logged in user is enrolled in the course 4. Backend returns course data 5. Frontend populates course page with course data 6. User clicks on a unit card to be expand and view the subtopics 7. Frontend changes state value of accordion to open the specified unit 8. Accordion re-renders with correct unit open

Screenshot of end state	See Fig. 2, Fig. 3
Exception Flow	1. Backend receives request and validates the logged in user is enrolled in the course 2. User is either not logged in or not enrolled; backend returns 403 (Forbidden)

Interaction	**Start adaptive testing for a subtopic**
Pre-condition	User is logged in and on a course page and is on the “Practice Problems” tab with a unit open displaying subtopics and their practice button
Screenshot of pre-condition	See Fig. 2
Action	User clicks on a practice button adjacent to a subtopic
Flow	1. Frontend redirects user to question loading page 2. Frontend sends out a request to backend with user’s bearer token “`/api/core/adaptive-test/next-question`” 3. Backend receives request, validates that the subtopic exists, the user is enrolled in the course (has permissions) 4. Backend returns a question with N options 5. Frontend populates question form with backend data
Screenshot of end state	See Fig. 5
Exception Flow	3. Backend receives request, validates that the subtopic exists, the user is enrolled in the course (has permissions) 4. User does not have permissions to the course 5. Backend returns 403 (Forbidden) 6. Frontend displays an alert informing the user they are not allowed to perform that action

- placement of items
  - Main navigation header we followed McMaster’s Brand Web Guidelines which helps to maintain a consistent experience across McMaster websites
- colour/font choices
  - We followed McMaster’s Brand Web Guidelines for all our colour and font choices.

Colour	**Role**
#FFFFFF	Background
#000000	Foreground
#FDBF57	Accent (used for Hover and borders)
#7A003C	Primary colour (Maroon)
#495965 (Opacity 50%)	Muted text

McMaster branding dictates the font must be Poppins (Regular for text, semibold for buttons and bold for headings)

Verification and Validation Plan

1. Frontend

Verification is done through unit tests and performance testing. Unit tests are used to ensure that UI components render correctly, handle user interactions as expected, and display appropriate states when data is loading or unavailable. Performance tests focus on confirming that page loads, navigation, and user interactions remain responsive and within acceptable latency limits during normal usage.

1.1 Question Page

The Question Page allows students to view practice questions, submit answers, and receive feedback during an adaptive testing session.

1.1.1 Unit Tests

- Verifying that questions and multiple-choice options are rendered correctly when retrieved from backend APIs.
- Confirm that user selections are correctly registered and visually reflected in the UI.
- Verifying that answer submission triggers the appropriate API request, and that feedback is being displayed.
- Ensuring that the page handles API failures correctly, such as displaying error or loading states when data is unavailable.

1.1.2 Performance Tests and Metrics

- Measuring question load time after navigation, which should remain under 1 second under normal conditions.
- Measuring response time between answer submission and feedback display, which should remain ≤ 5 seconds.
- Ensuring that the page remains responsive while waiting for backend responses.

1.2 Course, Unit, and Subtopic Details

This component allows students to navigate available courses, units, and subtopics before starting practice sessions.

1.2.1 Unit Tests

- Verify that enrolled courses are displayed correctly for authenticated users.
- Ensuring that selecting a course loads the corresponding units and subtopics.
- Verifying correct navigation behavior between hierarchical levels.
- Checking that appropriate UI messaging is shown when no data is available.

1.2.2 Performance Tests and Metrics

- Measuring initial page load time, which should remain under 2 seconds.
- Verifying that navigation between course levels (Course to Unit to Subtopic) occurs within 1 second.

1.3 Statistics Page

The Statistics Page shows a summary of the student's practice performance and progress.

1.3.1 Unit Tests

- Verifying that user statistics and visual summaries are requested and displayed.
- Verifying that statistics update correctly when new practice attempts are recorded.
- Verifying graceful handling of API errors or missing data.

1.3.2 Performance Tests and Metrics

- Measuring page load and render time, which should remain under 2 seconds.
- Ensuring visual elements render smoothly without any noticeable delay.

1.4 Frontend Validation

Frontend validation will be done by pilot testing with a group of student users to evaluate usability and educational effectiveness. Students will complete tasks such as navigating courses, answering questions, and reviewing their statistics. Feedback will be collected through a questionnaire that will focus on the overall usefulness of the interface. To further validate the practical impact of the system, pilot users may also reflect on how prepared they felt before and after using the platform. High-level comparisons between platform usage and course assessment outcomes may be reviewed. This validation approach provides qualitative evidence that the frontend supports effective exam preparation and a more guided study experience.

2. Backend

Tests are implemented using pytest and pytest-django, with Django REST Framework's API testing utilities. Performance tests confirm that the backend meets expected performance requirements. Locust is used to simulate concurrent users and measure load behavior, while Python's time library is used for lightweight runtime measurements of specific operations.

2.1 Question File Parser

2.1.1 Unit Tests

- Verifying that uploading a valid .docx file containing questions creates Question records with correct text content, difficulty values, and verification status.
- Verifying MCQs create Question Option records with correct option text.
- Verifying that embedded images in a .docx file generate Question Image records containing valid Base64-encoded images linked to the corresponding questions.
- Verifying that the Celery task errors when processing an unsupported file format.

- Verifying that upon successful completion of the Celery task parse_file(), all questions are stored in the DB

2.1.2 Performance Tests and Metrics

- The file parser performance is evaluated by measuring the Celery parse_file() runtime from file submission to completion of question extraction.
- The average processing time should be less than 30 seconds for a 50-question file with embedded images and less than 2 minutes for files containing up to 200 questions.
- Each measurement is repeated 10 times and averaged to ensure consistency.

2.2 Adaptive Test Framework

2.2.1 Unit Tests

- Verifying that submitting an answer correctly identifies whether the response is correct or incorrect and updates the user's ability to score accordingly.
- Verifying that requesting the next question returns a question within the appropriate difficulty range based on the user's current ability score.
- Verifying that a new UserTopicAbilityScore record is created when a user starts a test for a subtopic, they have not previously attempted.
- Verifying that when no questions are available the user is prompted with actions
- Verifying that questions with higher item information values are prioritized for selection

2.2.2 Performance Tests and Metrics

- Adaptive testing performance is evaluated through latency, by how quickly users receive the next question and correctness feedback after submitting an answer.
- Using Locust, the next question response time should be less than 3 seconds under normal load and less than 10 seconds under peak load with up to 100 concurrent users, while the submit response time should be less than 5 seconds under normal load and less than 15 seconds under peak load.

2.3 User Data Compilation

2.3.1 Unit Tests

- Verifying that submitting an answer creates a Question Attempt record with the correct user ID, question ID, selected option, and timestamp.
- Verifying that multiple attempts can be recorded when allowed.
- Verifying that correct answers increase User Topic Ability Score per IRT logic.
- Verifying that incorrect answers decrease ability score based on parameters.
- Verifying that final ability reflects cumulative performance across attempts.

2.3.2 Performance tests and metrics

- The user data compilation performance is measured through the latency of recording question attempts and the computation time of IRT ability score updates, timed using time library.

- Creating a single Question Attempt record should complete within 2 seconds and under peak load simulated with Locust at 100 concurrent submissions, each record should complete within 2 seconds.
- IRT ability score calculations should be completed in real time with an average runtime of less than 1 second, measured 50 times across varied parameters to confirm consistency.

3. Authentication, and Database

3.1 Postgres Database

Verification

3.1.1 Unit and Integration Tests

- Verifying database migrations apply cleanly across environments.
- Verifying CRUD operations for core models (Users, Questions, Question Attempts, Enrolments).
- Verifying cascade deletion removes dependent records (e.g., question options, images).
- Verifying failed transactions correctly rollback without partial writes.
- Verify concurrent writes do not produce duplicate or inconsistent records.

3.1.2 Performance Tests and Metrics

- Single write latency: Creating a Question Attempt record completes within ≤ 5s under normal load.
- Concurrent write latency: Under 100 concurrent submissions, writes complete within ≤ 2 s.
- Query latency: Common read queries (fetching questions, analytics) complete within ≤ 1s.
- Connection stability: No dropped database connections during sustained Locust load tests.

3.1.3 Reliability and Recovery Tests

- Verifying database container restarts without data loss.
- Verifying persistent volumes retain data across restarts.
- Verifying backups can be restored to a clean instance.
- Verifying database recovery time ≤ 5 minutes.

Validation

- Validating analytics accuracy by comparing frontend statistics with raw database aggregates.
- Validating database stability during pilot usage with real student activity.

3.2 Authentication and Access Control (M365 SSO)

Verification

3.2.1 Unit Tests

- OIDC User Creation
  - Verify first login creates a Mac*FASTUser* using OIDC username claim.
  - Verify users without email fallback to *sub* as a unique identifier.

        - Verifying repeated logins does not create duplicate users.
    - Access Control
        - Verifying requests without a token, expired token, or invalid token return 401 Unauthorized.
        - Verify non-admin access to admin endpoints returns 403 Forbidden.
        - Verifying role-based access restrictions are enforced consistently.

3.2.2 Performance Tests and Metrics

- Authentication performance is measured during the end-to-end login flow using Locust:
    - OIDC callback processing (excluding external provider latency) will be replicated to the user experience of McMaster's Avenue to Learn (A2L) login.
    - Session creation and role mapping: Completes within ≤ 2 seconds.
    - Under concurrent login scenarios, authentication behavior will be verified to match the stability and responsiveness observed in A2L during academic usage.

3.2.3 Security Tests

- Verifying tokens are never logged or stored persistently.
- Verifying token validation uses JWKS and handles key rotation correctly.
- Verifying all authentication traffic occurs over HTTPS.

Validation

- Validating authentication reliability with McMaster accounts in pilot deployment.
- Validating role correctness by confirming instructors and students see only authorized resources.
- Confirming authentication failures result in clear, user-facing error messages.

4. Cross-System Validation – Pilot Phase

- To validate MacFAST beyond technical correctness:
    - Conduct pilot testing with a small group of students.
    - Compare practice activity, adaptive progression, and perceived preparedness.
    - Where feasible, review anonymized trends between practice performance and course assessments (midterms/finals).

  This provides qualitative validation that the system infrastructure supports effective and uninterrupted learning, without making formal causal claims.

Appendix

UI/ UX

Fig. 1

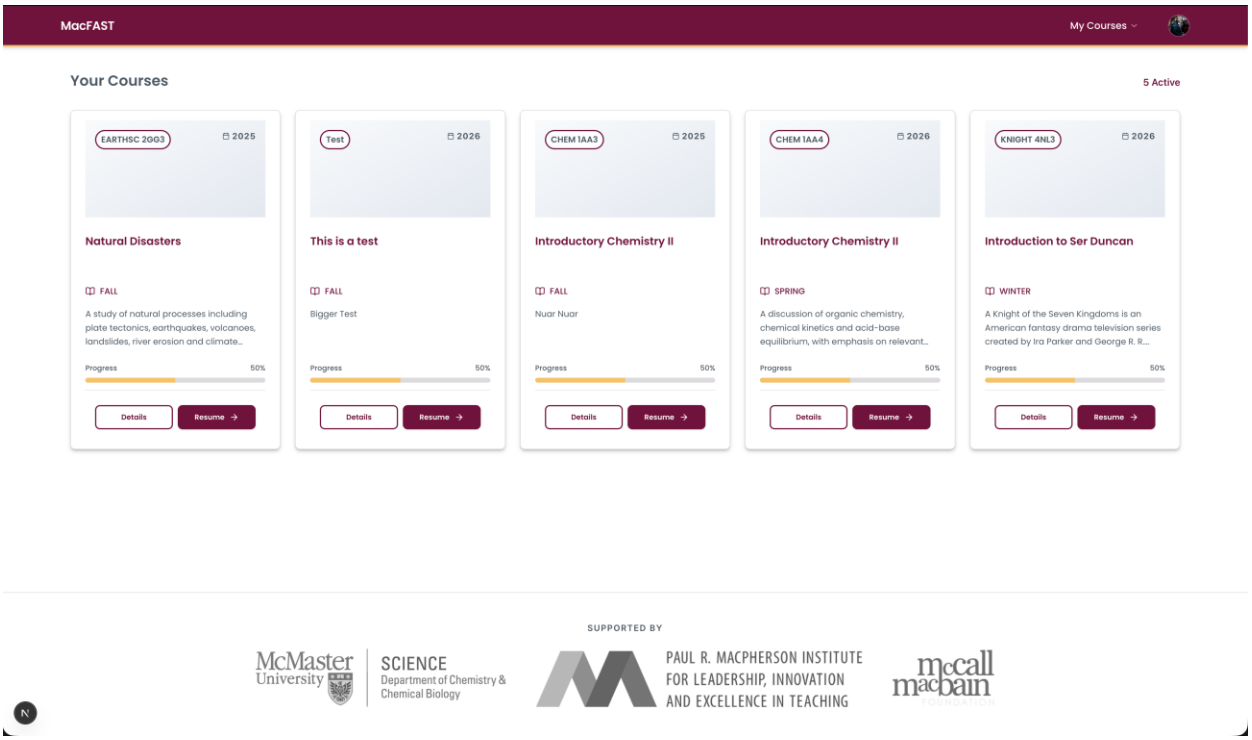
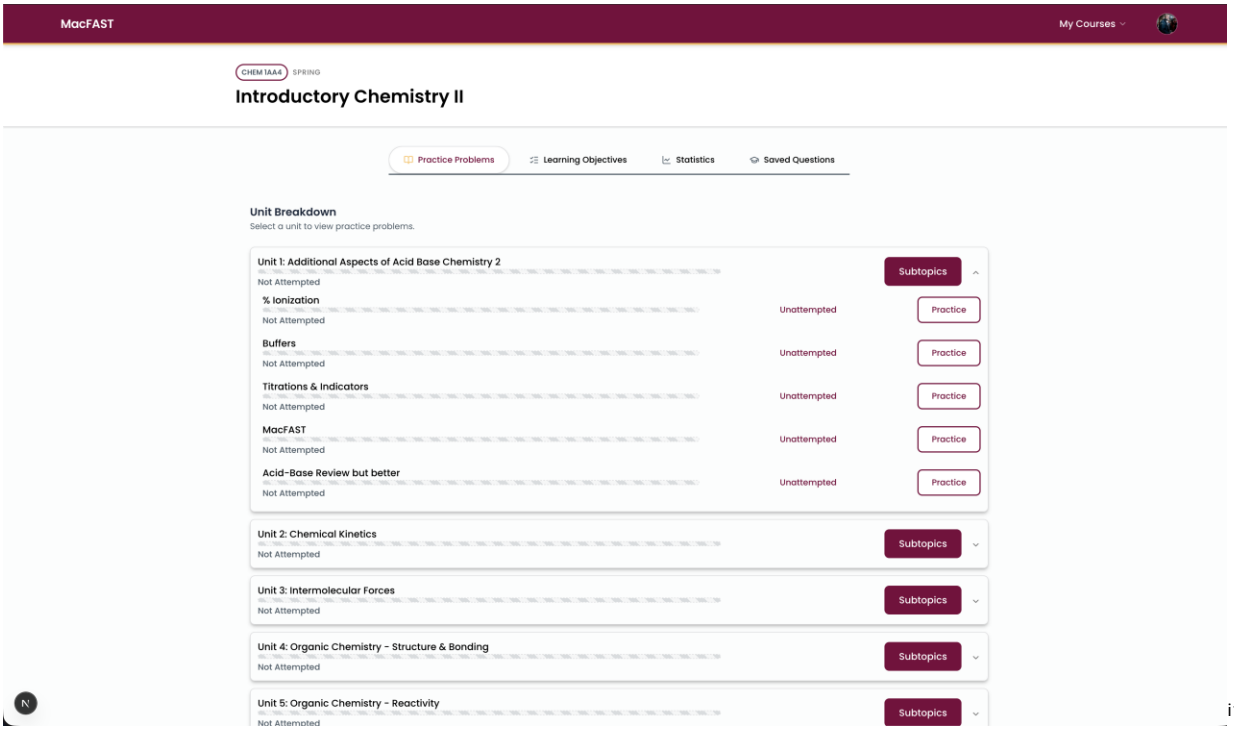
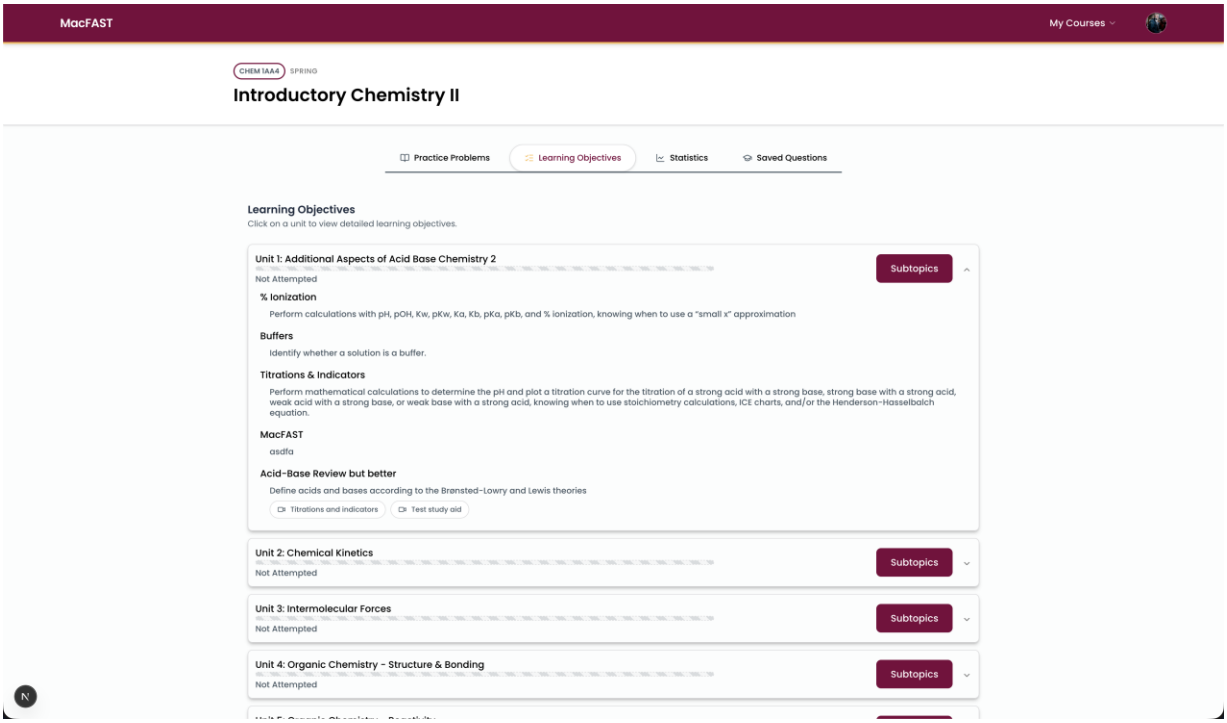


Fig. 2


MacFAST
My Courses
CHEM 1AA4
SPRING
Introductory Chemistry II
Practice Problems
Learning Objectives
Statistics
Saved Questions
Unit Breakdown
Select a unit to view practice problems.
Unit 1: Additional Aspects of Acid Base Chemistry 2
Not Attempted
Subtopics
% Ionization
Not Attempted
Unattempted
Practice
Buffers
Not Attempted
Unattempted
Practice
Titrations & Indicators
Not Attempted
Unattempted
Practice
MacFAST
Not Attempted
Unattempted
Practice
Acid-Base Review but better
Not Attempted
Unattempted
Practice
Unit 2: Chemical Kinetics
Not Attempted
Subtopics
Unit 3: Intermolecular Forces
Not Attempted
Subtopics
Unit 4: Organic Chemistry - Structure & Bonding
Not Attempted
Subtopics
Unit 5: Organic Chemistry - Reactivity
Subtopics

[1]

Fig. 3


MacFAST
My Courses
CHEM 1AA4
SPRING
Introductory Chemistry II
Practice Problems
Learning Objectives
Statistics
Saved Questions
Learning Objectives
Click on a unit to view detailed learning objectives.
Unit 1: Additional Aspects of Acid Base Chemistry 2
Not Attempted
Subtopics
% Ionization
Perform calculations with pH, pOH, Kw, pKw, Ka, Kb, pKa, pKb, and % ionization, knowing when to use a "small x" approximation
Buffers
Identify whether a solution is a buffer.
Titrations & Indicators
Perform mathematical calculations to determine the pH and plot a titration curve for the titration of a strong acid with a strong base, strong base with a strong acid, weak acid with a strong base, or weak base with a strong acid, knowing when to use stoichiometry calculations, ICE charts, and/or the Henderson-Hasselbalch equation.
MacFAST
asdfa
Acid-Base Review but better
Define acids and bases according to the Brønsted-Lowry and Lewis theories
Titrations and indicators
Test study aid
Unit 2: Chemical Kinetics
Not Attempted
Subtopics
Unit 3: Intermolecular Forces
Not Attempted
Subtopics
Unit 4: Organic Chemistry - Structure & Bonding
Not Attempted
Subtopics

1

Fig. 4

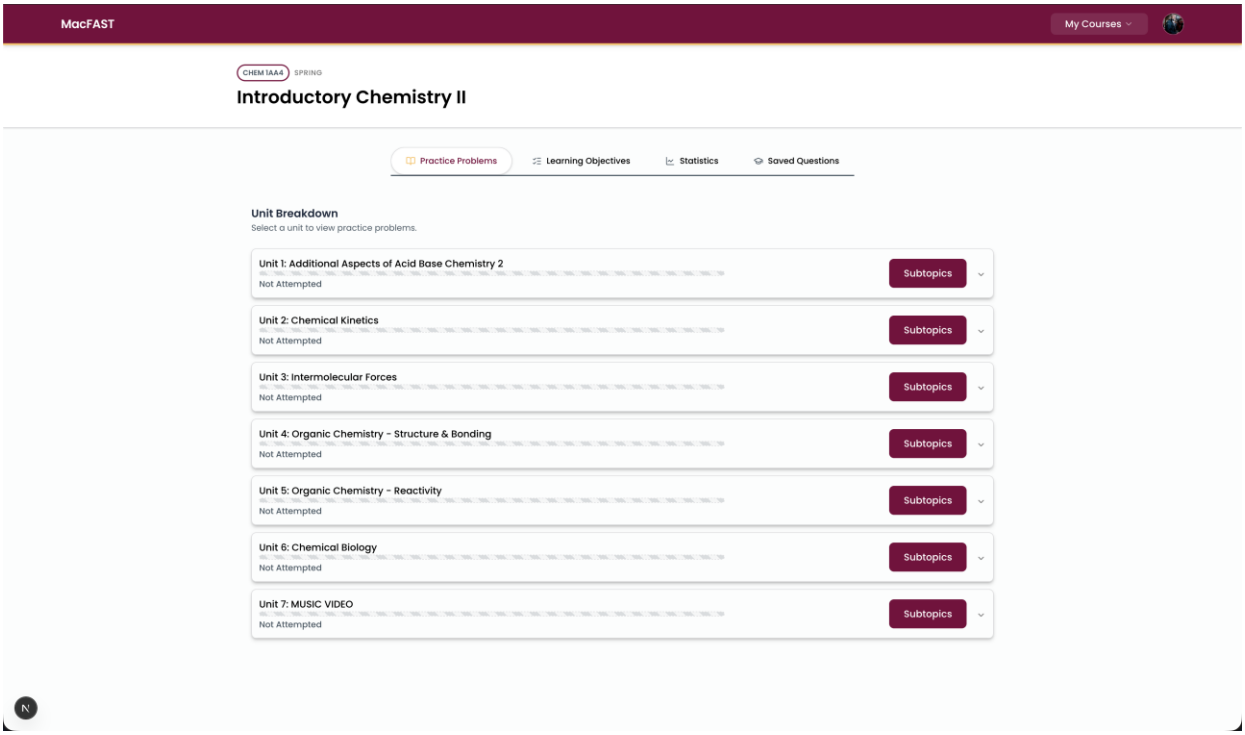
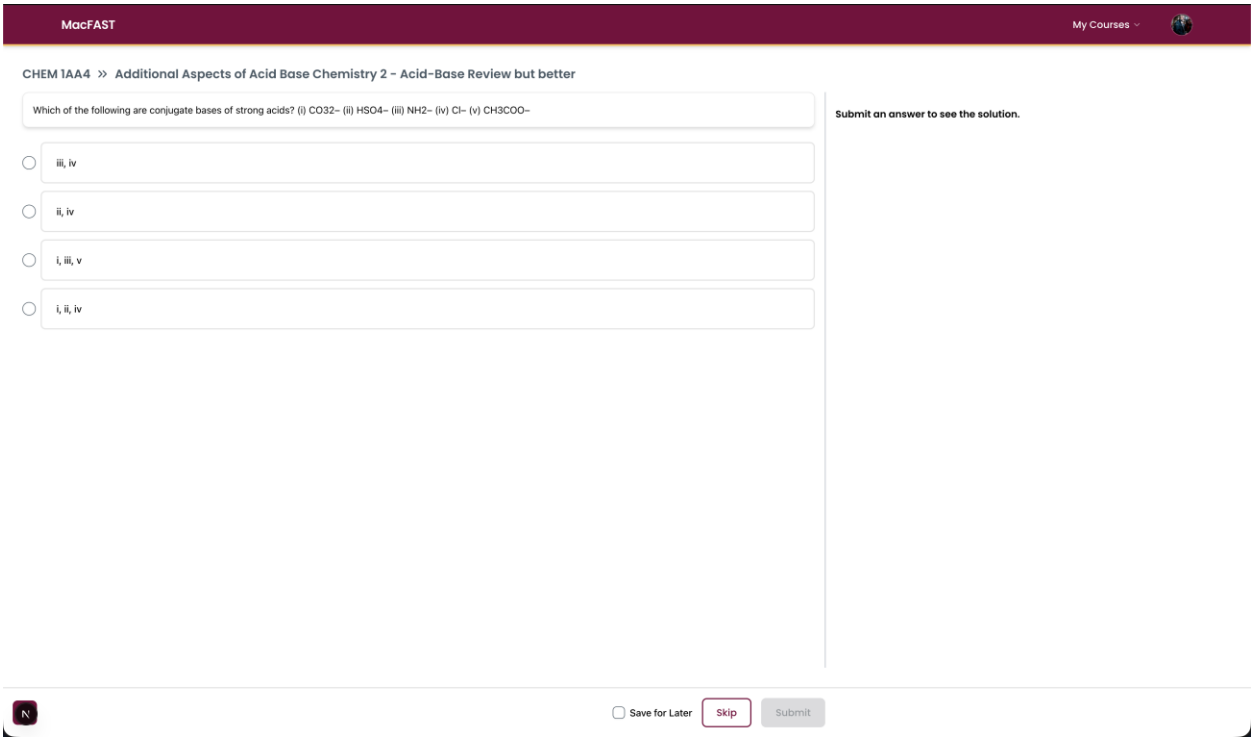

MacFAST
My Courses
CHEM 1AA4
SPRING
Introductory Chemistry II
Practice Problems
Learning Objectives
Statistics
Saved Questions
Unit Breakdown
Select a unit to view practice problems.
Unit 1: Additional Aspects of Acid Base Chemistry 2
Not Attempted
Subtopics
Unit 2: Chemical Kinetics
Not Attempted
Subtopics
Unit 3: Intermolecular Forces
Not Attempted
Subtopics
Unit 4: Organic Chemistry - Structure & Bonding
Not Attempted
Subtopics
Unit 5: Organic Chemistry - Reactivity
Not Attempted
Subtopics
Unit 6: Chemical Biology
Not Attempted
Subtopics
Unit 7: MUSIC VIDEO
Not Attempted
Subtopics

Fig. 5


MacFAST
My Courses
CHEM 1AA4 » Additional Aspects of Acid Base Chemistry 2 - Acid-Base Review but better
Which of the following are conjugate bases of strong acids? (i) CO32− (ii) HSO4− (iii) NH2− (iv) Cl− (v) CH3COO−
iii, iv
ii, iv
i, iii, v
i, ii, iv
Submit an answer to see the solution.
Save for Later
Skip
Submit

---

i